

Developing an Intelligent Job Recommendation System Using Semantic Retrieval and Explainable AI Techniques

Hussein Ali Al Awad* Dr. Khaled Fathi Omar*

*Master of Web Science, Syrian Virtual University, Damascus, Syria

Hussein Ali Al Awad: husseinite98@gmail.com

Dr. Khaled Fathi Omar: Kh.om.mail@gmail.com

Abstract—Online recruitment platforms require recommendation methods that can retrieve relevant job opportunities from large and heterogeneous collections of postings. Keyword-based search is efficient and interpretable, but it can miss relevant postings when equivalent roles are expressed with different terms. This study presents a metadata-only job recommendation system that combines TF-IDF lexical matching, Sentence-BERT semantic retrieval, query-aware filtering, optional Cross-Encoder re-ranking, and explanation generation. The system uses structured fields, including job title, company name, location, seniority level, job function, employment type, and industry, without relying on full job descriptions or user interaction histories. Experiments on a cleaned LinkedIn job posting dataset containing 31,262 records show that the best hybrid configuration achieved Precision@10 of 0.8032 and nDCG@10 of 0.9496. Under the internal evaluation protocol, Cross-Encoder re-ranking improved Precision@10 from 0.7896 to 0.7948 and nDCG@10 from 0.9666 to 0.9739. These results indicate that lexical and semantic retrieval can be combined effectively for explainable job recommendation when only structured metadata is available.

Index Terms—Job recommendation system, semantic retrieval, explainable artificial intelligence, TF-IDF, Sentence-BERT, Cross-Encoder, hybrid ranking, information retrieval, natural language processing.

I. INTRODUCTION

Online recruitment platforms have become important channels for matching job seekers with employment opportunities. These platforms commonly contain postings that differ by job title, company, location, seniority level, employment type, job function, and industry. As the number of postings increases, effective retrieval becomes necessary because users must identify relevant opportunities from large and heterogeneous collections.

Many recruitment search interfaces still rely on lexical matching. This approach is computationally efficient and transparent, and it remains useful for exact constraints such as location, seniority level, and employment type. However, lexical search can under-rank relevant postings when equivalent roles are expressed with different surface forms. For example, postings titled “software engineer”, “application developer”, and “backend engineer” may be relevant to a query for “software developer”, even when exact term overlap is limited.

Dense semantic retrieval provides a complementary retrieval signal. Transformer-based language models, including BERT

[1] and Sentence-BERT [2], can encode short texts into vector representations that support semantic similarity search. Such models are useful when job titles and metadata contain related meanings but different terms. Nevertheless, semantic similarity alone may be insufficient for job search because explicit constraints, including work mode, location, seniority, and employment type, should be preserved.

The gap addressed in this study is the design of an accurate and explainable job recommendation method for a restricted metadata-only setting. In this setting, full job descriptions, user profiles, click logs, application histories, and manually labeled relevance judgments are unavailable. The proposed system addresses this gap by combining TF-IDF lexical matching, Sentence-BERT semantic candidate retrieval, weighted hybrid ranking, query-aware filtering, optional Cross-Encoder re-ranking, and metadata-based explanation generation.

The main contributions of this paper are as follows:

- A metadata-only job recommendation framework that does not require full descriptions or historical user interactions.
- A hybrid retrieval model that combines TF-IDF lexical similarity and Sentence-BERT semantic embeddings.
- A semantic candidate generation strategy based on normalized dense vectors and nearest-neighbor retrieval.
- Query-aware filtering for employment type, seniority level, location, and remote, hybrid, or onsite work mode.
- An explainability layer that reports matched terms, applied filters, and metadata-based evidence.
- An empirical evaluation using Precision@10 and nDCG@10 on a cleaned LinkedIn job posting dataset.

II. RELATED WORK

Recommendation systems are commonly categorized as collaborative filtering, content-based filtering, or hybrid methods [3]. Collaborative filtering uses interaction histories such as ratings, clicks, or applications. Although this approach can be effective on mature platforms, it is limited in cold-start scenarios where new users or items have little historical data. Content-based methods use item attributes and are therefore more suitable when interaction data is unavailable.

Information retrieval methods provide a foundation for content-based recommendation. Classical lexical models such

as TF-IDF are widely used because they preserve exact term evidence and can be implemented efficiently [4]. In job recommendation, exact terms remain important because many user queries contain explicit requirements, including locations, employment types, seniority levels, and work-mode constraints.

Transformer-based models have improved semantic representation and ranking. The transformer architecture introduced attention-based sequence modeling [5], and BERT introduced bidirectional contextual representations for natural language processing [1]. Sentence-BERT adapted transformer representations for efficient sentence-level similarity search [2]. Dense vector search can be scaled using nearest-neighbor indexing approaches, including GPU-oriented similarity search methods such as FAISS [6]. Cross-Encoder ranking models jointly process a query and candidate document, often improving ranking quality at higher computational cost [7], [8]. For this reason, they are typically applied to a limited set of top-ranked candidates rather than the full collection.

Explainability is also relevant to recommender systems, particularly in domains where recommendations may influence important decisions. Explanations may be generated through matched features, score decomposition, rules, or concise textual rationales [9]. In employment recommendation, metadata-based explanations are practical because visible fields such as job title, function, seniority, employment type, and location can be directly connected to user intent.

III. PROBLEM STATEMENT

This study addresses the problem of recommending relevant job postings using only structured metadata. The available fields include job title, company name, location, hiring status, date, seniority level, job function, employment type, and industry. Full job descriptions, user profiles, click logs, application histories, and human relevance labels are not assumed to be available.

This setting is challenging for several reasons. Job metadata is short, role titles are inconsistent across organizations, abbreviations are common, and **user queries may combine semantic intent with strict constraints**. A query such as “remote junior data analyst London” includes a target role, work mode, seniority level, and location. A useful recommender should capture semantic similarity for the role while preserving exact constraints when they are explicitly stated.

The main technical challenges are summarized as follows:

- Lexical retrieval may miss semantically related postings with different wording.
- Semantic retrieval may return broadly related postings that violate explicit constraints.
- Metadata-only records provide less textual evidence than full job descriptions.
- Neural recommendation components require interpretable outputs to support user trust.
- Offline evaluation is difficult when human relevance labels are unavailable.

IV. PROPOSED METHODOLOGY

The proposed methodology follows a pipeline-oriented design. Raw job postings are validated and cleaned, and each posting is converted into a composite metadata document. Two indexes are then created: a sparse TF-IDF matrix and a dense Sentence-BERT embedding matrix. During recommendation, the query is encoded using both representations. Semantic nearest-neighbor retrieval generates candidate postings, and lexical and semantic scores are normalized and combined through a weighted hybrid scoring function. Query-aware filters, duplicate suppression, company-level diversification, optional Cross-Encoder re-ranking, and explanation generation are then applied.

A. Metadata Document Construction

Each job posting is represented by a composite metadata document:

$$d_i = [t_i, c_i, l_i, s_i, f_i, e_i, r_i], \quad (1)$$

where t_i is the job title, c_i is the company name, l_i is the location, s_i is the seniority level, f_i is the job function, e_i is the employment type, and r_i is the industry. The title is repeated because it is usually the strongest visible signal in job search.

B. Lexical Similarity

The lexical component represents metadata documents and queries using TF-IDF. The TF-IDF weight of term t in document d is defined as [4]:

$$\text{tfidf}(t, d) = \text{tf}(t, d) \times \log \left(\frac{N}{\text{df}(t) + 1} \right), \quad (2)$$

where N is the number of documents and $\text{df}(t)$ is the number of documents containing term t . Lexical similarity is computed as a sparse dot product between the query vector and candidate document vectors.

C. Semantic Similarity

The semantic component uses the Sentence Transformer model `sentence-transformers/all-MiniLM-L6-v2`.

Each document d_i and query q is encoded into normalized dense vectors $\mathbf{v}_{d_i}$ and $\mathbf{v}_q$. Semantic similarity is computed as:

$$s_{\text{sem}}(q, d_i) = \mathbf{v}_q^\top \mathbf{v}_{d_i}. \quad (3)$$

Because the embeddings are normalized, the inner product is equivalent to cosine similarity.

D. Hybrid Ranking

The lexical and semantic scores are min-max normalized before fusion:

$$\hat{s}(x) = \frac{s(x) - \min(s)}{\max(s) - \min(s) + \epsilon}. \quad (4)$$

The hybrid score is then computed as:

$$s_{\text{hybrid}}(q, d_i) = 0.4 \hat{s}_{\text{sem}}(q, d_i) + 0.6 \hat{s}_{\text{lex}}(q, d_i). \quad (5)$$

TABLE I
FUNCTIONAL LAYERS OF THE PROPOSED JOB RECOMMENDATION
ARCHITECTURE.

Layer	Main Function	Output
Data ingestion	Reads and validates structured job metadata	Validated records
Preprocessing	Normalizes text, expands abbreviations, and removes noisy titles	Cleaned metadata
Indexing	Builds TF-IDF vectors and Sentence-BERT embeddings	Sparse and dense indexes
Retrieval	Retrieves semantic candidates using vector similarity	Candidate list
Hybrid ranking	Combines lexical, semantic, metadata, and optional re-ranking scores	Ranked recommendations
Explainability	Reports matched terms, filters, and metadata evidence	Transparent output

The lexical component is assigned a larger weight because exact terms in titles, locations, work modes, and seniority levels are important in job search. The semantic component remains necessary because it expands retrieval beyond exact keyword overlap.

E. Cross-Encoder Re-ranking

When re-ranking is enabled, the top candidates are processed as query-document pairs using cross-encoder/ms-marco-MiniLM-L-6-v2. The final score is computed as:

$$s_{\text{final}}(q, d_i) = \alpha \hat{s}_{\text{rerank}}(q, d_i) + (1 - \alpha) s_{\text{hybrid}}(q, d_i) + b_i, \quad (6)$$

where $\alpha = 0.7$ and b_i is an optional metadata bonus assigned when a candidate shares strong metadata evidence with the seed job in the evaluation setting.

F. Implementation Details

The implementation used Python with pandas, NumPy, scikit-learn, SciPy, joblib, sentence-transformers, PyTorch, and Streamlit. Data preparation reads the LinkedIn Excel file, validates required columns, normalizes text, expands common abbreviations, removes noisy job titles, and saves the cleaned dataset as an artifact. Index training builds a TF-IDF matrix and a Sentence-BERT embedding matrix over the composite metadata documents. Runtime recommendation loads the stored artifacts, retrieves semantic candidates using NearestNeighbors with cosine distance, computes lexical and semantic scores, applies hybrid fusion, optionally performs Cross-Encoder re-ranking, removes duplicates, limits repeated companies, and returns explanation fields.

V. SYSTEM ARCHITECTURE

The architecture is organized as a layered retrieval and recommendation pipeline. Each layer has a specific function and passes structured outputs to the next layer. This modular design supports maintainability, reproducibility, and controlled experimentation.

VI. RETRIEVAL PIPELINE

The retrieval pipeline combines semantic candidate generation with hybrid ranking. Semantic retrieval is applied first because it can identify related postings even when exact lexical overlap is limited. Lexical evidence is then reintroduced during ranking to preserve exact matching for important query constraints.

A. Query-Aware Filtering and Diversification

The system extracts explicit filters from the query when possible. Supported filters include employment type, seniority level, location hints, and work mode. If the extracted filters remove all candidates, the system falls back to the unfiltered candidate set to avoid returning an empty result list.

The final ranking stage also applies duplicate suppression using a tuple of normalized job title, company, and location. A company-level cap limits repeated results from the same employer. These rules improve result diversity without changing the core retrieval model.

VII. EXPLAINABLE AI LAYER

The explainability component is designed to make recommendations understandable to end users. Instead of exposing only a numerical score, the system reports evidence that links the user query to the recommended posting. This form of explanation is consistent with feature-based explanation strategies for recommender systems [9].

The explanation layer includes four forms of evidence:

- *Keyword overlap*: shared terms between the user query and the composite job document.
- *Filter explanation*: extracted constraints such as remote, hybrid, junior, full-time, or a location term.
- *Metadata evidence*: matches in job function, industry, seniority level, or employment type.
- *Ranking evidence*: indication of whether a result was mainly supported by lexical matching, semantic similarity, or neural re-ranking.

For example, for the query “remote junior data analyst London”, a recommended job may include an explanation showing matched keywords such as “data”, “analyst”, “remote”, and “London”, together with applied filters for work mode, seniority, and location. This explanation does not fully interpret the internal embedding space, but it provides practical transparency based on metadata visible to users.

VIII. DATASET AND PREPROCESSING

The evaluation used a LinkedIn job posting dataset. The raw dataset contained 31,597 records. After cleaning and validation, 31,262 valid records remained. The system used only structured metadata fields: job title, company name, location, hiring status, date, seniority level, job function, employment type, and industry.

The preprocessing pipeline included missing-value handling, whitespace normalization, job-title normalization, abbreviation expansion, and removal of noisy records. Examples of abbreviation expansion include ML to machine learning, AI

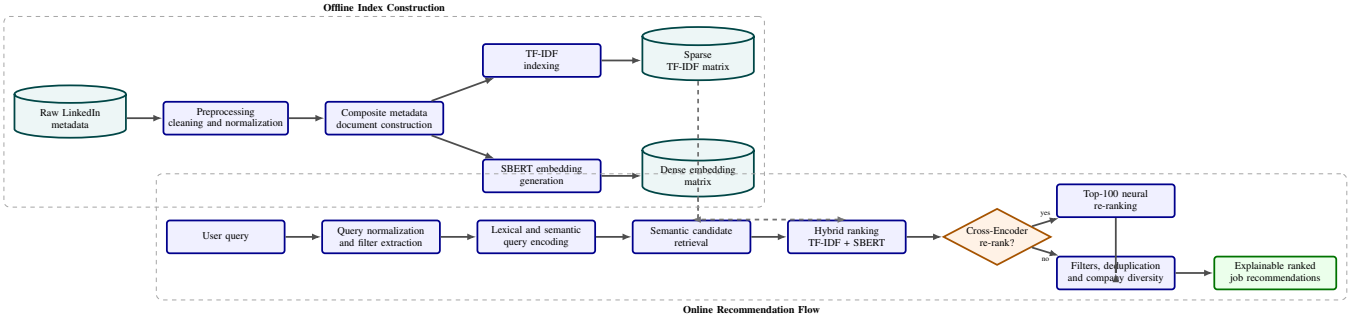


Fig. 1. Overall architecture of the proposed intelligent job recommendation system.

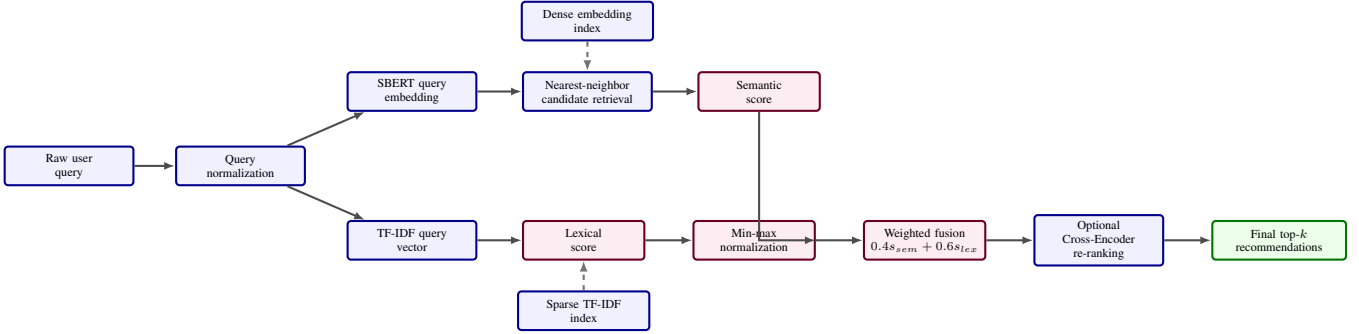


Fig. 2. Hybrid retrieval pipeline combining semantic candidate generation and lexical-semantic ranking.

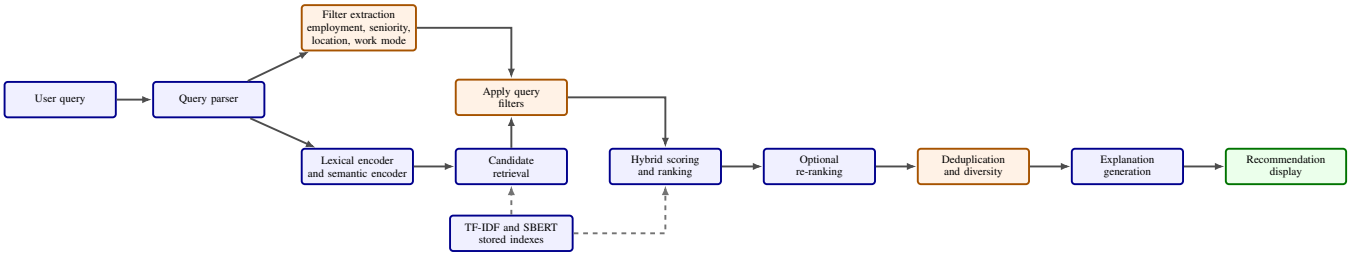


Fig. 3. Online query flow from user input to explainable job recommendations.

to artificial intelligence, SWE and SDE to software engineer, QA to quality assurance, PM to product manager, and MLE to machine learning engineer.

IX. EXPERIMENTAL EVALUATION

The evaluation used an internal metadata-based relevance protocol because human relevance judgments and user interaction logs were unavailable. Seed jobs were sampled from the dataset, and their normalized job titles were used as queries. Retrieved jobs were compared with the seed job using metadata consistency.

Relevance labels were assigned as follows: 3 for the same normalized job title, 2 for the same job function or industry, 1 for the same seniority level or employment type, and 0 otherwise. Precision@10 considered labels 2 and 3 as relevant. nDCG@10 was used to evaluate ranking quality with graded relevance.

Precision@10 is defined as:

$$\text{Precision@10} = \frac{|\{d_i \in R_{10} : \text{rel}(d_i) \geq 2\}|}{10}, \quad (7)$$

where R_{10} is the set of the top 10 returned jobs.

Discounted cumulative gain at rank k is computed as:

$$\text{DCG@}k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}. \quad (8)$$

The normalized version is:

$$\text{nDCG@}k = \frac{\text{DCG@}k}{\text{IDCG@}k}, \quad (9)$$

where IDCG@ k is the ideal DCG obtained by sorting results by true graded relevance.

A. Hybrid Retrieval Results

Table V reports the hybrid retrieval results across different candidate sizes and weighting settings. The best reported

TABLE II
DATASET FIELDS USED BY THE METADATA-ONLY RECOMMENDATION SYSTEM.

Field	Role in the system
Job title	Primary role signal and strongest lexical evidence
Company name	Employer metadata and duplicate-control feature
Location	Geographic and work-location matching
Hiring status	Structured posting status metadata
Date	Temporal metadata retained from the source dataset
Seniority level	Career-level filtering and relevance grading
Job function	Functional category for matching and evaluation
Employment type	Full-time, part-time, contract, internship, and related filters
Industry	Domain-level evidence for matching and relevance grading

TABLE III
DATASET SIZE BEFORE AND AFTER PREPROCESSING.

Stage	Records	Description
Raw dataset	31,597	Original LinkedIn job postings
Cleaned dataset	31,262	Valid records after preprocessing
Removed records	335	Invalid, noisy, or unusable records

setting used 250 semantic candidates with semantic weight 0.4 and lexical weight 0.6, achieving Precision@10 of 0.8032 and nDCG@10 of 0.9496.

B. Cross-Encoder Re-ranking Results

Table VI compares the baseline hybrid ranking with the optional Cross-Encoder re-ranking stage. Re-ranking was applied to the top 100 candidates using $\alpha = 0.7$. The Cross-Encoder produced modest improvements in both metrics under the internal evaluation protocol.

X. DISCUSSION

The results show that a hybrid retrieval design is suitable for metadata-only job recommendation under the evaluation protocol used in this study. Larger semantic candidate sets improved the likelihood that relevant postings were available to the ranking stage. At the same time, configurations with stronger lexical weighting performed best. This result is consistent with the short and structured nature of the metadata, where exact terms in job titles, locations, seniority levels, and employment types remain informative.

The Cross-Encoder re-ranking stage improved both Precision@10 and nDCG@10, although the gains were moderate. This outcome is expected because the baseline hybrid model already performed strongly under the metadata-derived relevance protocol. The benefit of re-ranking is its more detailed query-candidate interaction, while its drawback is increased computational cost [8].

The explanation layer provides practical transparency by showing matched keywords and applied filters. The approach does not claim to fully explain dense embedding behavior.

TABLE IV
INTERNAL RELEVANCE GRADING PROTOCOL USED FOR OFFLINE EVALUATION.

Grade	Condition	Interpretation
3	Same normalized job title	Highly relevant
2	Same job function or same industry	Relevant
1	Same seniority level or same employment type	Weakly related
0	None of the above metadata matches	Not relevant

TABLE V
HYBRID RETRIEVAL PERFORMANCE UNDER DIFFERENT CANDIDATE SIZES AND WEIGHTING CONFIGURATIONS.

Candidates	Sem.	Lex.	P@10	nDCG@10
80	0.7	0.3	0.7624	0.9360
80	0.6	0.4	0.7676	0.9388
80	0.5	0.5	0.7780	0.9392
80	0.4	0.6	0.7884	0.9432
150	0.7	0.3	0.7640	0.9365
150	0.6	0.4	0.7776	0.9413
150	0.5	0.5	0.7844	0.9420
150	0.4	0.6	0.7984	0.9478
250	0.7	0.3	0.7680	0.9393
250	0.6	0.4	0.7832	0.9389
250	0.5	0.5	0.7936	0.9454
250	0.4	0.6	0.8032	0.9496

Instead, it exposes metadata-based evidence that is directly connected to visible job attributes and user query terms.

A. Limitations

This study has several limitations. First, the system uses structured metadata only and does not include full job descriptions, skill requirements, salary, education level, or company descriptions. Second, the evaluation uses heuristic relevance labels derived from metadata consistency rather than human judgments or real user interactions. Third, the models are pre-trained general-purpose models and were not fine-tuned on a job-specific relevance dataset. Fourth, Cross-Encoder re-ranking improves quality but increases latency and computational cost. Finally, the dataset represents a fixed snapshot of job postings, while operational recruitment platforms require continuous updates as jobs are posted, modified, and closed.

XI. CONCLUSION

This paper presented an intelligent job recommendation system that combines semantic retrieval, lexical matching, explainable AI techniques, and optional neural re-ranking. The system was designed for a metadata-only setting where full descriptions and user interaction histories are unavailable. The proposed pipeline uses TF-IDF for exact lexical evidence, Sentence-BERT for semantic candidate retrieval, weighted hybrid scoring for ranking, query-aware filters for explicit constraints, and explanation fields for transparency.

Experiments on a cleaned LinkedIn job dataset showed that the best hybrid configuration achieved Precision@10 of 0.8032 and nDCG@10 of 0.9496. Cross-Encoder re-ranking further improved Precision@10 from 0.7896 to 0.7948 and

TABLE VI
BASELINE HYBRID RANKING VERSUS CROSS-ENCODER RE-RANKING.

Configuration	P@10	nDCG@10	Notes
Baseline hybrid	0.7896 $\pm$ 0.2896	0.9666 $\pm$ 0.1051	No Cross-Encoder
Cross-Encoder	0.7948 $\pm$ 0.2946	0.9739 $\pm$ 0.1046	Top 100 candidates
Delta	+0.0052	+0.0072	Rerank minus baseline

nDCG@10 from 0.9666 to 0.9739 under the internal evaluation protocol. These findings support the conclusion that effective and interpretable metadata-only job recommendation is feasible when preprocessing, retrieval, ranking, filtering, and evaluation are carefully controlled.

Future work should include human relevance judgments, real user interaction data, full job descriptions, skill extraction, learning-to-rank models [10], fairness analysis, and deployment-oriented latency evaluation.

REFERENCES

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 4171–4186.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 2019, pp. 3982–3992.
- [3] F. Ricci, L. Rokach, and B. Shapira, Eds., *Recommender Systems Handbook*, 2nd ed. New York, NY, USA: Springer, 2015.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [7] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, “BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,” in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.
- [8] J. Lin, R. Nogueira, and A. Yates, “Pretrained transformers for text ranking: BERT and beyond,” *Synthesis Lectures on Human Language Technologies*, vol. 14, no. 4, pp. 1–325, 2021.
- [9] Y. Zhang and X. Chen, “Explainable recommendation: A survey and new perspectives,” *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020.
- [10] C. Burges, T. Shaked, E. Renshaw, A. Lazier, M. Deeds, N. Hamilton, and G. Hullender, “Learning to rank using gradient descent,” in *Proceedings of the 22nd International Conference on Machine Learning*, 2005, pp. 89–96.